

Jetpack Compose第三章第四节教学讲义

(Material Design主题与UI规范)

各位同学，我们开发APP时，除了实现功能，还需要让UI美观、统一且符合用户习惯——这就离不开Material Design。Material Design是Google推出的一套跨平台UI设计规范，它定义了颜色、文本、形状、组件交互等一系列标准，而Compose天生就深度集成了Material Design 3（最新版本）。这节课咱们就从“定义专属主题”开始，逐步学习如何在Compose中应用Material Design规范，让你的APP既好看又专业！

一、先搞懂：Material Design是什么？

很多新手会把Material Design和“好看的UI”画等号，但它的核心是“有质感的设计系统”——就像现实世界的物体有颜色、形状、光影一样，Material Design让UI组件也具备类似的“物理属性”，比如按钮被点击时会有“按压凹陷”的反馈。

Material Design 3的核心要素

Compose中使用的Material Design 3（简称MD3），核心围绕4个要素构建，这也是我们后续开发的重点：

- **颜色 (Color)**：包含主色、辅助色、中性色等，确保APP颜色统一且符合 accessibility（无障碍）规范；
- **排版 (Typography)**：定义文本的字体、大小、粗细等样式，让不同层级的文本清晰易读；
- **形状 (Shape)**：控制组件的圆角、边框等，比如按钮的圆角大小、卡片的边框样式；
- **组件 (Component)**：基于以上要素的预制组件，如Button、Card、TextField等，开箱即用。

Compose通过“主题 (Theme)” 将这四个要素整合起来，我们只需定义一次主题，所有组件就能自动应用对应的样式，实现全局UI统一。

二、核心实战：定义专属Material主题

新建Compose项目时，Android Studio会自动生成默认主题代码（在ui/theme目录下），但实际开发中我们需要根据产品设计稿修改为专属主题。主题定义的核心是配置“颜色、排版、形状”三大核心要素。

步骤1：理解默认主题结构

默认主题包含三个核心文件，它们的职责分工明确：

文件名称	核心作用
Color.kt	定义主题的所有颜色值，如主色、辅助色、中性色
Type.kt	定义文本排版样式，如标题、正文、提示文本的字体样式
Shape.kt	定义组件的形状样式，如按钮、卡片、输入框的圆角大小
Theme.kt	整合颜色、排版、形状，生成最终的MaterialTheme

步骤2：自定义主题颜色（Color.kt）

MD3的颜色系统有严格的规范，核心是“主色（Primary）”“辅助色（Secondary）”和“中性色（Neutral）”，每个颜色又包含不同的色调（如PrimaryLight、PrimaryDark），以适应不同场景。

实战：定义电商APP主题色

假设产品设计稿要求主色为橙色（代表活力），辅助色为蓝色（代表信任），中性色为深灰/浅灰（文本和背景），代码如下：

Code block

```
1  import androidx.compose.ui.graphics.Color
2
3  // 1. 主色（橙色系）：用于按钮、标题、重点强调区域
4  val Orange50 = Color(0xFFFFF3E0) // 主色浅色调
5  val Orange100 = Color(0xFFFFE0B2)
6  val Orange500 = Color(0xFFFF9800) // 主色基准色（核心）
7  val Orange700 = Color(0xFFF57C00) // 主色深色调
8  val Orange900 = Color(0xFFE65100)
9
10 // 2. 辅助色（蓝色系）：用于标签、提示、次要按钮
11 val Blue50 = Color(0xFFE3F2FD)
12 val Blue500 = Color(0xFF2196F3) // 辅助色基准色
13 val Blue700 = Color(0xFF1976D2)
14
15 // 3. 中性色（灰色系）：用于文本、背景、分割线
16 val Neutral50 = Color(0xFFFFFFFF) // 背景色（白色）
17 val Neutral100 = Color(0xFFF5F5F5) // 浅背景
18 val Neutral700 = Color(0xFF616161) // 正文文本色
19 val Neutral900 = Color(0xFF212121) // 标题文本色
20
21 // 4. 功能色：用于状态提示（成功、错误、警告）
```

```
22 val Success500 = Color(0xFF4CAF50)
23 val Error500 = Color(0xFFFF44336)
24 val Warning500 = Color(0xFFFFFC107)
```

步骤3：自定义文本排版（Type.kt）

MD3的排版系统基于“字体家族（FontFamily）”和“文本样式（TextStyle）”，我们需要为不同层级的文本（如页面标题、卡片标题、正文、提示语）定义统一的样式，确保阅读体验一致。

实战：定义电商APP文本样式

Code block

```
1  import androidx.compose.ui.text.TextStyle
2  import androidx.compose.ui.text.font.Font
3  import androidx.compose.ui.text.font.FontFamily
4  import androidx.compose.ui.text.font.FontWeight
5  import androidx.compose.ui.unit.sp
6  import com.example.yourapp.R // 替换为你的APP包名
7
8  // 1. 引入自定义字体（可选，无自定义字体可直接用系统默认）
9  // 假设assets/fonts目录下有PingFangSC.ttf字体文件
10 val PingFangFontFamily = FontFamily(
11     Font(R.font.pingfangsc, FontWeight.Normal),
12     Font(R.font.pingfangsc_bold, FontWeight.Bold)
13 )
14
15 // 2. 定义文本样式（按层级划分）
16 val AppTypography = androidx.compose.material3.Typography(
17     // 页面标题：粗体、24sp
18     headlineLarge = TextStyle(
19         fontFamily = PingFangFontFamily,
20         fontWeight = FontWeight.Bold,
21         fontSize = 24.sp,
22         color = Neutral900
23     ),
24     // 卡片标题：粗体、18sp
25     titleMedium = TextStyle(
26         fontFamily = PingFangFontFamily,
27         fontWeight = FontWeight.Bold,
28         fontSize = 18.sp,
29         color = Neutral900
30     ),
31     // 正文文本：常规、16sp
32     bodyLarge = TextStyle(
33         fontFamily = PingFangFontFamily,
```

```

34         fontWeight = FontWeight.Normal,
35         fontSize = 16.sp,
36         color = Neutral700
37     ),
38     // 提示文本：常规、14sp
39     labelMedium = TextStyle(
40         fontFamily = PingFangFontFamily,
41         fontWeight = FontWeight.Normal,
42         fontSize = 14.sp,
43         color = Neutral700
44     ),
45     // 价格文本：粗体、16sp、主色
46     bodyMedium = TextStyle(
47         fontFamily = PingFangFontFamily,
48         fontWeight = FontWeight.Bold,
49         fontSize = 16.sp,
50         color = Orange500
51     )
52 )

```

注意：引入自定义字体时，需在main目录下创建assets/fonts文件夹，放入字体文件，并在build.gradle中配置assets目录（Android Studio通常会自动配置）。

步骤4：自定义组件形状（Shape.kt）

形状主要控制组件的“圆角半径（CornerRadius）”和“边框（Border）”，统一的形状样式能让APP视觉更协调。比如电商APP常用“小圆角按钮”“中等圆角卡片”。

实战：定义电商APP组件形状

Code block

```

1  import androidx.compose.foundation.shape.CornBasedShape
2  import androidx.compose.foundation.shape.CornerRadiusShape
3  import androidx.compose.foundation.shape.RoundedCornerShape
4
5  // 按圆角大小划分形状样式
6  val AppShapes = androidx.compose.material3.Shapes(
7      // 小圆角：用于按钮、输入框（4dp）
8      small = RoundedCornerShape(4.dp),
9      // 中等圆角：用于卡片、列表项（8dp）
10     medium = RoundedCornerShape(8.dp),
11     // 大圆角：用于弹窗、头像（16dp）
12     large = RoundedCornerShape(16.dp)
13 )

```

步骤5：整合主题（Theme.kt）

Theme.kt的核心作用是将前面定义的颜色、排版、形状整合为 `MaterialTheme` 组件，通过 `CompositionLocal`（上节课学的隐式传参）将主题信息提供给整个APP的组件树。

实战：整合自定义主题

Code block

```
1  import androidx.compose.foundation.isSystemInDarkTheme
2  import androidx.compose.material3.MaterialTheme
3  import androidx.compose.material3.darkColorScheme
4  import androidx.compose.material3.lightColorScheme
5  import androidx.compose.runtime.Composable
6
7  // 1. 定义浅色主题配色方案
8  private val LightColorScheme = lightColorScheme(
9      primary = Orange500,          // 主色基准色
10     primaryContainer = Orange100, // 主色容器色（如卡片背景）
11     secondary = Blue500,          // 辅助色基准色
12     background = Neutral50,       // 页面背景色
13     surface = Neutral100,         // 组件表面色（如卡片）
14     onPrimary = Neutral50,        // 主色上的文本色（如按钮文本）
15     onBackground = Neutral900,    // 背景上的文本色（如页面标题）
16     error = Error500              // 错误色
17     // 可根据需要补充其他颜色
18 )
19
20 // 2. 定义深色主题配色方案（可选，适配夜间模式）
21 private val DarkColorScheme = darkColorScheme(
22     primary = Orange300,
23     primaryContainer = Orange700,
24     secondary = Blue300,
25     background = Neutral900,
26     surface = Neutral800,
27     onPrimary = Neutral900,
28     onBackground = Neutral50,
29     error = Error300
30 )
31
32 // 3. 对外暴露的主题组件
33 @Composable
34 fun ECommerceTheme(
35     darkTheme: Boolean = isSystemInDarkTheme(), // 自动判断是否为深色模式
36     content: @Composable () -> Unit // 接收APP的根组件
37 ) {
38     // 根据模式选择颜色方案
```

```

39     val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme
40
41     // 提供主题：所有子组件都能通过MaterialTheme获取样式
42     MaterialTheme(
43         colorScheme = colorScheme,
44         typography = AppTypography, // 自定义排版
45         shapes = AppShapes,         // 自定义形状
46         content = content            // 渲染APP根组件
47     )
48 }

```

步骤6：应用主题到APP

在APP的根组件（通常是MainActivity的setContent中）包裹主题组件，整个APP的所有子组件就能自动应用主题样式：

Code block

```

1  import androidx.appcompat.app.AppCompatActivity
2  import android.os.Bundle
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.tooling.preview.Preview
6
7  class MainActivity : AppCompatActivity() {
8      override fun onCreate(savedInstanceState: Bundle?) {
9          super.onCreate(savedInstanceState)
10         setContent {
11             // 包裹主题组件
12             ECommerceTheme {
13                 // APP的根组件（如首页）
14                 HomeScreen()
15             }
16         }
17     }
18 }
19
20 // 首页组件（自动应用主题）
21 @Composable
22 fun HomeScreen() {
23     // 背景色自动使用主题的背景
24     androidx.compose.foundation.layout.Box(
25         modifier = androidx.compose.foundation.layout.fillMaxSize()
26     ) {
27         // 文本样式自动使用主题的headlineLarge
28         Text(

```

```

29         text = "电商首页",
30         style = MaterialTheme.typography.headlineLarge,
31         modifier = androidx.compose.foundation.layout.padding(16.dp)
32     )
33 }
34 }
35
36 @Preview(showBackground = true)
37 @Composable
38 fun HomeScreenPreview() {
39     ECommerceTheme {
40         HomeScreen()
41     }
42 }

```

三、主题应用：颜色、文本、形状的实战技巧

定义好主题后，核心是在组件中正确使用主题样式——Compose的Material组件（如Button、Card、TextField）会自动应用主题，我们也可以手动获取主题样式自定义组件。

1. 主题颜色的使用技巧

使用主题颜色时，优先通过 `MaterialTheme.colorScheme` 获取，而非直接使用Color常量，这样能确保主题切换（如浅色/深色模式）时颜色同步变化。

实战：在组件中使用主题颜色

Code block

```

1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.layout.Button
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.material3.Card
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.ui.Modifier
9  import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.unit.dp
11
12 @Composable
13 fun ThemeColorDemo() {
14     // 获取主题颜色
15     val colorScheme = MaterialTheme.colorScheme
16

```

```

17     Column(modifier = Modifier.padding(16.dp)) {
18         // 1. Material组件自动应用主题色（推荐）
19         Button(onClick = { /* 点击事件 */ }) {
20             Text(text = "立即购买")
21             // 按钮背景: colorScheme.primary
22             // 按钮文本: colorScheme.onPrimary
23         }
24
25         // 2. 自定义组件手动应用主题色
26         Card(
27             modifier = Modifier
28                 .padding(top = 16.dp)
29                 .fillMaxWidth(),
30             // 卡片背景: colorScheme.surface
31             // 卡片圆角: MaterialTheme.shapes.medium（自动应用）
32         ) {
33             Text(
34                 text = "商品名称: 智能手机",
35                 style = MaterialTheme.typography.titleMedium,
36                 modifier = Modifier.padding(16.dp),
37                 color = colorScheme.onSurface // 卡片上的文本色
38             )
39         }
40
41         // 3. 功能色的使用（错误提示）
42         Text(
43             text = "库存不足",
44             style = MaterialTheme.typography.labelMedium,
45             modifier = Modifier.padding(top = 8.dp),
46             color = colorScheme.error // 主题错误色
47         )
48     }
49 }

```

2. 主题文本的处理技巧

文本样式的核心是“按层级使用主题定义的Typography”，避免在每个Text组件中单独设置fontSize、fontWeight等，确保全局文本风格统一。

实战：文本样式的正确使用

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.Text

```



```
4 import androidx.compose.runtime.Composable
5 import androidx.compose.ui.Modifier
6 import androidx.compose.ui.unit.dp
7
8 @Composable
9 fun ThemeTextDemo() {
10     val typography = MaterialTheme.typography
11
12     Column(modifier = Modifier.padding(16.dp)) {
13         // 页面标题：使用headlineLarge
14         Text(
15             text = "商品详情",
16             style = typography.headlineLarge
17         )
18
19         // 商品标题：使用titleMedium
20         Text(
21             text = "超薄全面屏智能手机",
22             style = typography.titleMedium,
23             modifier = Modifier.padding(top = 8.dp)
24         )
25
26         // 商品价格：使用bodyMedium（自定义的价格样式）
27         Text(
28             text = "¥2999",
29             style = typography.bodyMedium,
30             modifier = Modifier.padding(top = 4.dp)
31         )
32
33         // 商品描述：使用bodyLarge
34         Text(
35             text = "6.7英寸超清大屏，5000mAh大电池，支持66W快充",
36             style = typography.bodyLarge,
37             modifier = Modifier.padding(top = 16.dp)
38         )
39
40         // 提示文本：使用labelMedium
41         Text(
42             text = "限时优惠，下单立减200元",
43             style = typography.labelMedium,
44             modifier = Modifier.padding(top = 8.dp),
45             color = MaterialTheme.colorScheme.secondary
46         )
47     }
48 }
```

3. 主题形状的处理技巧

形状样式通过 `MaterialTheme.shapes` 获取，主要用于自定义组件的圆角设置，Material组件会自动应用主题形状（如Card默认使用medium形状）。

实战：形状样式的应用

Code block

```
1  import androidx.compose.foundation.layout.Button
2  import androidx.compose.foundation.layout.Column
3  import androidx.compose.foundation.layout.padding
4  import androidx.compose.foundation.shape.CircleShape
5  import androidx.compose.material3.Card
6  import androidx.compose.material3.OutlinedTextField
7  import androidx.compose.material3.Text
8  import androidx.compose.runtime.Composable
9  import androidx.compose.runtime.getValue
10 import androidx.compose.runtime.mutableStateOf
11 import androidx.compose.runtime.remember
12 import androidx.compose.runtime.setValue
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.unit.dp
15
16 @Composable
17 fun ThemeShapeDemo() {
18     val shapes = MaterialTheme.shapes
19     var inputText by remember { mutableStateOf("") }
20
21     Column(modifier = Modifier.padding(16.dp)) {
22         // 1. Material组件自动应用主题形状
23         Card(
24             modifier = Modifier.fillMaxWidth(),
25             // 自动使用shapes.medium（中等圆角）
26         ) {
27             Text(text = "主题形状卡片", modifier = Modifier.padding(16.dp))
28         }
29
30         // 2. 输入框使用小圆角（shapes.small）
31         OutlinedTextField(
32             value = inputText,
33             onChange = { inputText = it },
34             label = { Text("请输入内容") },
35             modifier = Modifier
36                 .padding(top = 16.dp)
37                 .fillMaxWidth(),
38             shape = shapes.small
```

```

39         )
40
41         // 3. 自定义圆形按钮（结合主题形状和颜色）
42         Button(
43             onClick = { /* 点击事件 */ },
44             modifier = Modifier
45                 .padding(top = 16.dp)
46                 .size(48.dp),
47             shape = CircleShape // 圆形（也可自定义为主题形状）
48         ) {
49             Text(text = "√")
50         }
51     }
52 }

```

四、进阶技巧：主题的扩展与适配

实际开发中，主题还需要支持“动态切换”“多品牌适配”等场景，这里介绍两个实用技巧。

1. 支持手动切换浅色/深色模式

默认主题通过 `isSystemInDarkTheme()` 判断系统模式，我们可以通过状态管理让用户手动切换：

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Switch
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.runtime.getValue
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.remember
8  import androidx.compose.runtime.setValue
9
10 @Composable
11 fun ThemeSwitchDemo() {
12     // 状态：控制是否为深色模式
13     var isDarkMode by remember { mutableStateOf(false) }
14
15     // 手动传入darkTheme参数
16     ECommerceTheme(darkTheme = isDarkMode) {
17         Column(modifier = androidx.compose.foundation.layout.padding(16.dp)) {
18             // 主题切换开关
19             androidx.compose.foundation.layout.Row(
20                 verticalAlignment =
21                 androidx.compose.ui.Alignment.CenterVertically

```

```

21         ) {
22             Text(text = "深色模式")
23             Switch(
24                 checked = isDarkMode,
25                 onCheckedChange = { isDarkMode = it },
26                 modifier =
27                 androidx.compose.foundation.layout.padding(start = 8.dp)
28             )
29         }
30         Text(
31             text = "主题切换演示",
32             style = MaterialTheme.typography.headlineLarge,
33             modifier = androidx.compose.foundation.layout.padding(top =
34             16.dp)
35         )
36     }
37 }

```

2. 多品牌主题适配（如多渠道APP）

如果需要为不同渠道（如淘宝、天猫）定制不同主题，可以定义主题配置类，动态生成主题：

Code block

```

1  // 1. 定义主题配置类
2  data class ThemeConfig(
3      val primaryColor: Color,
4      val secondaryColor: Color,
5      val appName: String
6  )
7
8  // 2. 预设不同品牌的配置
9  val TaobaoConfig = ThemeConfig(
10     primaryColor = Color(0xFFFF4400),
11     secondaryColor = Color(0xFF00AAEE),
12     appName = "淘宝"
13 )
14
15 val TmallConfig = ThemeConfig(
16     primaryColor = Color(0xFFFF2E4D),
17     secondaryColor = Color(0xFF00CC99),
18     appName = "天猫"
19 )
20
21 // 3. 动态生成主题

```

```
22  @Composable
23  fun BrandTheme(
24      config: ThemeConfig,
25      content: @Composable () -> Unit
26  ) {
27      val colorScheme = lightColorScheme(
28          primary = config.primaryColor,
29          secondary = config.secondaryColor,
30          // 其他颜色基于配置生成
31      )
32
33      MaterialTheme(
34          colorScheme = colorScheme,
35          typography = AppTypography,
36          shapes = AppShapes,
37          content = content
38      )
39  }
40
41  // 4. 使用不同品牌主题
42  @Composable
43  fun TaobaoScreen() {
44      BrandTheme(config = TaobaoConfig) {
45          Text(text = TaobaoConfig.appName)
46      }
47  }
```

五、小节回顾：核心知识点速查表

核心内容	关键API/文件	核心技巧
Material Design 3核心	颜色、排版、形状、组件	基于规范定义，避免随意设置样式
主题定义	Color.kt、Type.kt、Shape.kt、Theme.kt	按文件分工，统一管理样式
颜色使用	MaterialTheme.colorScheme	优先用主题颜色，而非直接用Color常量
文本处理	MaterialTheme.typography	按层级使用预设样式，确保统一
形状处理	MaterialTheme.shapes	Material组件自动应用，自定义组件手动引用

主题切换	ECommerceTheme(darkTheme = ...)	通过状态控制darkTheme参数实现切换
------	---------------------------------	-----------------------

开发避坑指南

- 1. 不要在组件中硬编码颜色、字体大小等，必须通过主题获取，便于后续维护和切换；
- 2. 定义颜色时，遵循MD3的“色阶规范”（如50、100、500、700），确保颜色搭配协调；
- 3. 文本样式优先使用主题的Typography，避免每个Text组件单独设置样式；
- 4. 开发初期就定义好主题，不要等到UI杂乱后再重构，否则成本极高。

六、课后作业：动手实现“社交APP主题”

需求：基于Material Design 3，为社交APP定义专属主题，并实现主题切换功能，整合本节课知识点：

- 1. 定义主题颜色 (Color.kt) :
 主色：蓝色系（代表社交的信任与连接）；
- 2. 辅助色：粉色系（代表互动与活力）；
- 3. 中性色：深灰/浅灰（文本和背景）；
- 4. 功能色：成功（绿色）、错误（红色）、提醒（黄色）。
- 5. 定义文本排版 (Type.kt) :
 引入自定义字体（如思源黑体）；
- 6. 定义层级：聊天标题（20sp粗体）、用户名（16sp粗体）、聊天内容（16sp常规）、时间提示（12sp常规）。
- 7. 定义形状 (Shape.kt) :
 小圆角（4dp）：输入框、按钮；
- 8. 大圆角（20dp）：聊天气泡；
- 9. 圆形（50%）：用户头像。
- 10. 整合主题 (Theme.kt) : 支持浅色/深色模式；
- 11. 实现聊天界面组件：
 用户头像（圆形）；
- 12. 聊天气泡（大圆角，区分自己和他人的气泡颜色）；
- 13. 输入框（小圆角，带发送按钮）；
- 14. 主题切换开关（手动切换浅色/深色模式）。
- 15. 测试要求：切换主题时，所有组件的颜色、文本、形状都能同步更新，无硬编码样式。

提示：聊天气泡的颜色可基于主题的primary和secondary色，自己的气泡用主色，他人的气泡用中性色。重点练习主题的完整定义流程和组件中主题样式的应用，确保理解主题与组件的关联机制，以及主题切换的实现原理。

（注：文档部分内容可能由 AI 生成）